

保健統計学実習

第5日目

- 第13回 AI Deep learning演習(2) *Python Collaboratory使用
 第14回 AI Deep learning演習(3) *Python Collaboratory使用
 第15回 まとめ

滋賀医科大学NCD疫学研究センター
 医療統計学部門

原田 亜紀子
 (aharada@belle.shiga-med.ac.jp)

1

講義・演習スケジュール

回	日	内容
1	8/27(木)	1 R, EZR の使い方, データセットの読み込み, 頻度集計 2 記述統計, 相関 3 エクセルの基礎, Python 設定, 基本コード
2	8/28(金)	4 仮説検定の基礎, カイ2乗検定, マクネマー検定 5 t 検定, Wilcoxon 検定, サンプルサイズ設計 6 調査データ解析
3	9/1(火)	7 分散分析 8 重回帰分析 9 ロジスティック回帰, Poisson 回帰
4	9/2(水)	10 生存時間解析 11 クラスター分析 12 AI Deep learning 演習(1) *Python Collaboratory 使用
5	9/4(金)	13 AI Deep learning 演習(2) *Python Collaboratory 使用 14 AI Deep learning 演習(3) *Python Collaboratory 使用 15 解析実習まとめ(復習・課題の時間)

2

2

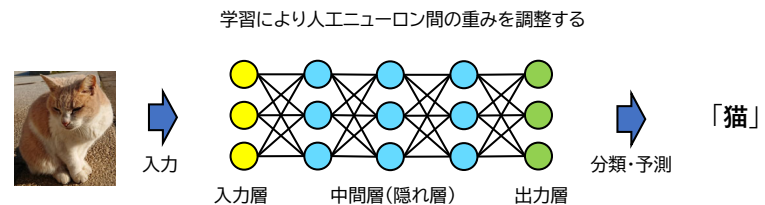
本日の講義内容

- ニューラルネットワークによる多クラス分類
 - 損失関数、活性化関数
 - 深層学習用ライブラリTensorFlow、Keras
 - 手書き数字の画像の分類
 - 練習問題 Fashion item画像の分類
- 畳み込みニューラルネットワークによるクラス分類
 - 食事分類(CNN)
- YOLO(物体検出)
 - 食事画像から各種メニューの抽出
 - アノテーション体験

3

深層学習

深層学習では、人工ニューロンによるネットワークを何層にも重ね合わせ、対象の特徴を自動で抽出する。



層を深く積み重ねることで、複雑な入力データの分類や数値の予測を行える

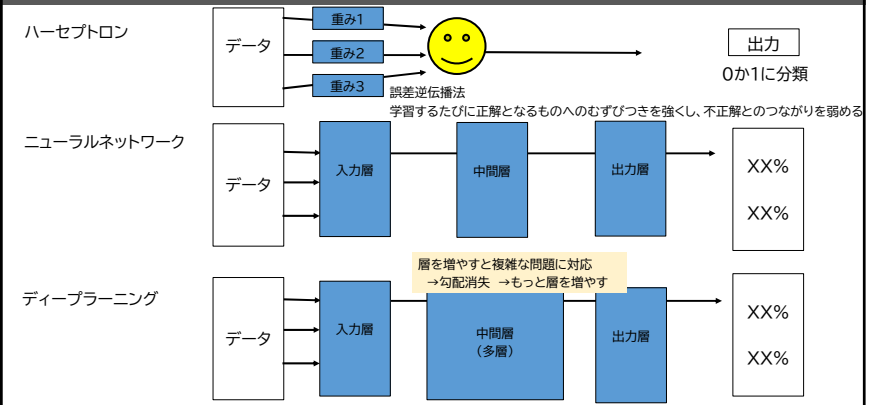
4

深層学習の応用例

1. 画像認識
 - 顔認証システム
 - 不良品検知
 - 画像検索
 - パン屋のレジサービス
 - 医療の画像認識
 - 自動運転
 - 画像生成
2. 音声認識
 - 音声アシスト
 - 音声翻訳
 - 議事録サービス
3. 自然言語処理
 - 自動翻訳
 - chatbot
 - ChatGPT, Gemini, Claude...

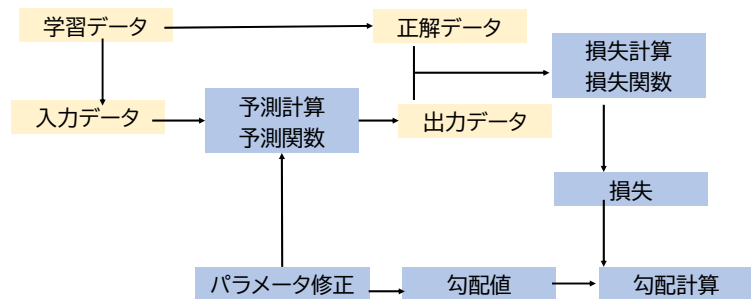
5

パーセプトロン・ニューラルネットワーク・ディープラーニング



6

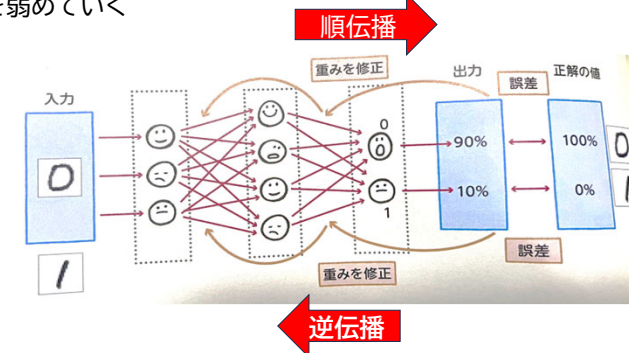
原理



7

なぜ自分で学習できるのか？

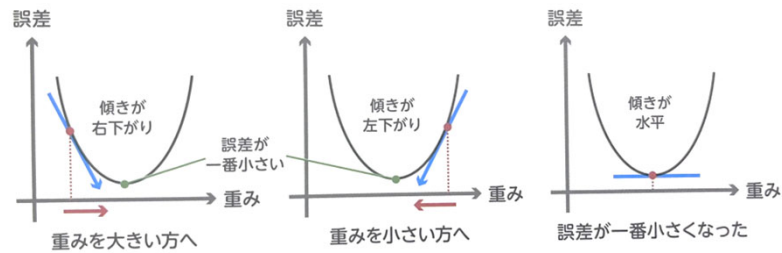
誤差を使って、正解となるつながりを強化し、不正解となるつながりを弱めていく



8

8

間違いを修正するためにどうやって重みを設定していくか？

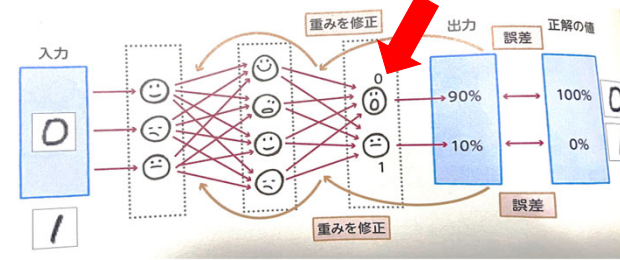


9

9

しきい値・活性化関数

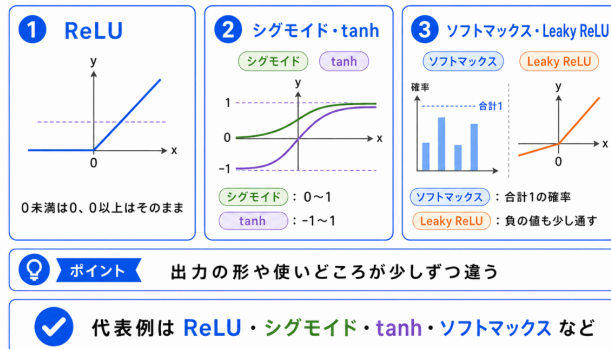
しきい値を超えたら活動が活性化し出力



10

10

活性化関数

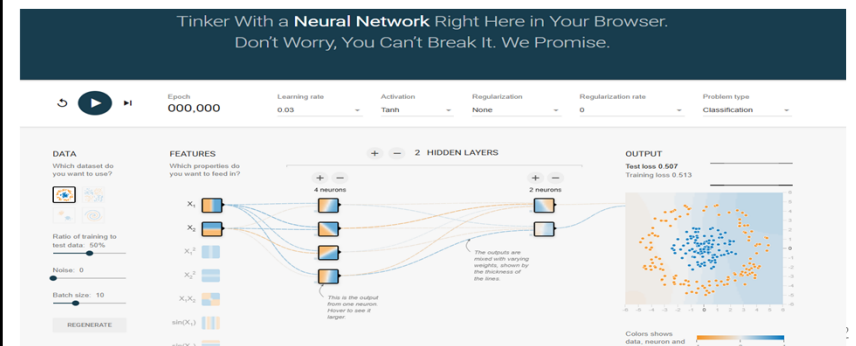


11

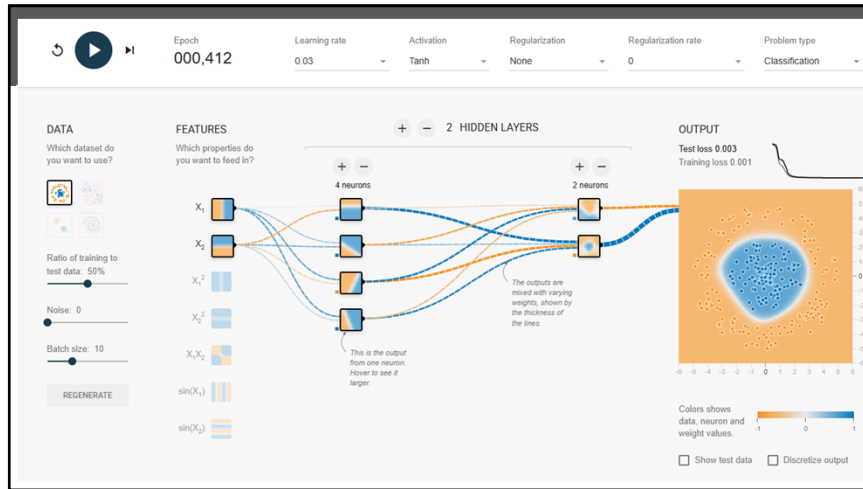
11

playground.tensorflow <https://playground.tensorflow.org/>

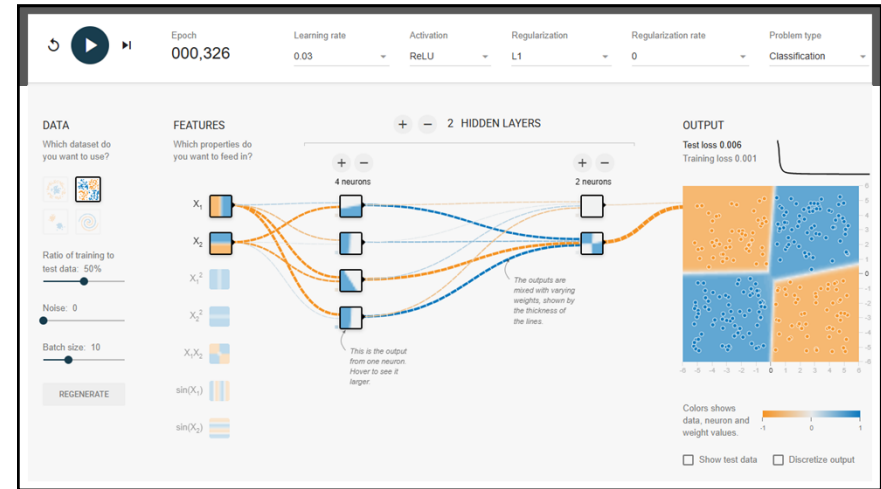
・ニューラルネットワークがどのように学習するかを見てみよう



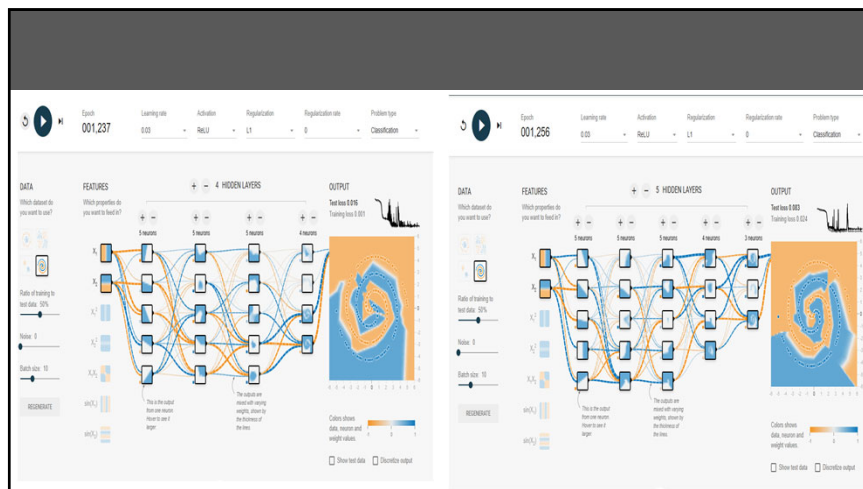
12



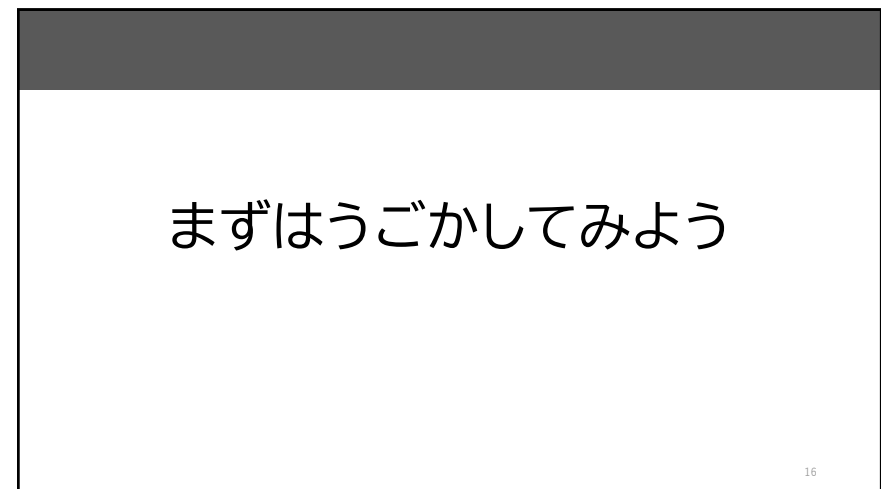
13



14



15



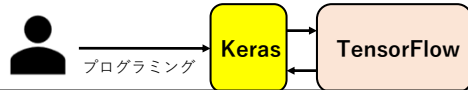
16

まずはうごかしてみよう

深層学習用のライブラリ

この講義では、TensorFlowとKerasというライブラリを用いて深層学習を学んでいく。

- TensorFlowはGoogleが開発している深層学習用のライブラリ。Meta(Facebook)が開発しているPyTorchとならんでよく使われるライブラリ。
- Kerasは深層学習用のライブラリをより簡単に使えるようにするためのインターフェイスを提供するGoogleが開発しているライブラリ。TensorFlowで直接プログラムを書くより簡単。TensorFlow以外のライブラリとも連携できる。



17

手書き文字の分類

Kerasにはサンプルデータセットとして、MNISTデータベース(Modified National Institute of Standards and Technology database)の手書きの0から9までの数字の画像が含まれている。

多層のニューラルネットワークを使って、これらの画像の分類を行う。



サンプルプログラムmnist.ipynbを実行していく。

18

忘れずに！！colaboratoryでのGPUの利用

時間制限はあるが、google colaboratoryの環境ではGPUを使って計算ができる。ディープラーニングでは、GPUを使うことで大幅に実行時間を短縮できる。



19

コードの説明

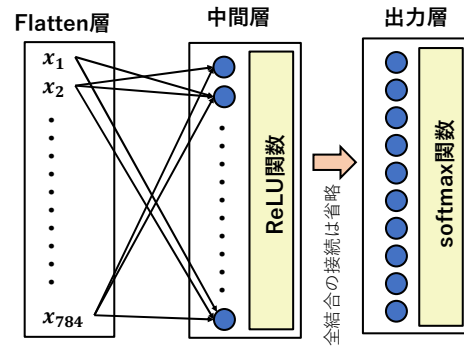
20

20

ニューラルネットワークと深層学習

今回作成したのはニューロンの層を二つ重ねただけのモデルだが、画像の分類という複雑な処理が可能となった。

深層学習では中間層の数を増やし、より複雑な問題を解けるようにしている。



21

ライブラリとデータの読み込み

TensorFlowに付属したKerasを使う。

TensorFlowをtfという名称で読み込む

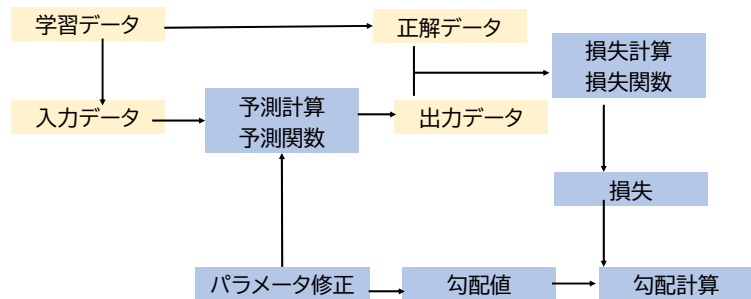
```
import tensorflow as tf
from tensorflow.keras import datasets
```

(x_train, y_train), (x_test, y_test) = datasets.mnist.load_data()

訓練用データ テスト用データ MNISTのデータを読み込む

22

原理



23

データのサイズと次元の確認

各次元の要素数を確認できる。

```
print("訓練用画像の枚数と画素数")
print(x_train.shape)

print("テスト用画像の枚数と画素数")
print(x_test.shape)
```

訓練用画像は60000枚 訓練用画像の枚数と画素数
(60000, 28, 28)

テスト用画像は10000枚 テスト用画像の枚数と画素数
(10000, 28, 28)

画像の画素数は28×28

24

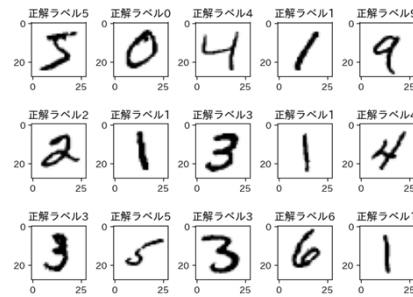
画像を表示

訓練用画像の一部を表示してみる。

`ax.imshow(x_train[i], cmap='Greys')`

グレースケール(白黒)で表示

正解のラベルがついている



matplotlibのimshowを使うと2次元配列を画像として表示できる

25

カテゴリ変数のone-hotエンコーディング

機械学習では、本来数値ではない変数をプログラムで扱いやすくするために数値のラベルに置き換えて扱うことがある。

このような質的なデータについての変数をカテゴリ変数と呼ぶ。

例えば、血液型を次のように数値に置き換えて扱うことができる。

A型	→ 0		A型	→ [1, 0, 0, 0]
B型	→ 1		B型	→ [0, 1, 0, 0]
AB型	→ 2		AB型	→ [0, 0, 1, 0]
O型	→ 3		O型	→ [0, 0, 0, 1]

ベクトル形式で表現することもある

こちらの表現を**one-hotエンコーディング**といい、ディープラーニングでもよく用いられる。

26

画像ラベルのone-hotエンコーディング

手書き数字の画像には画像の種類を表す0から9までのラベルがつけられている。

これはカテゴリ変数で数値の大小に意味はない。

Kerasのto_categoricalを用いてone-hotエンコーディングを行う。

```
from tensorflow.keras.utils import to_categorical

y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)
```

元のラベルが保存された配列を渡す

全部で10クラスのラベルがある

27

モデルの作成

Kerasのsequential(直列)モデルを作成する。

Sequentialをインポートしておく

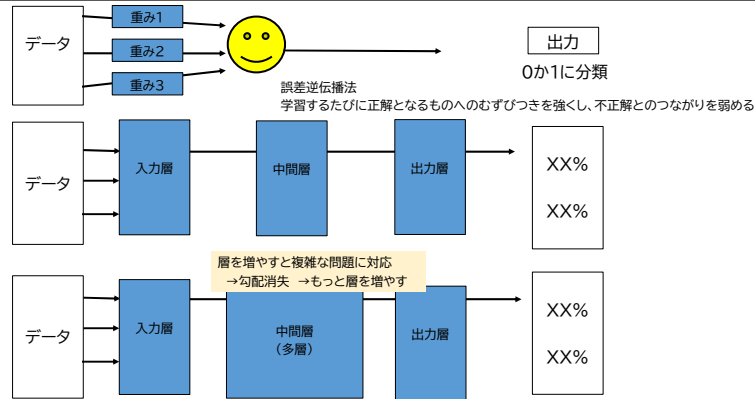
```
from tensorflow.keras.models import Sequential

model = Sequential()
```

Sequentialでモデルを作成する。
この時点ではまだニューロンの層がない空っぽの状態。

28

パーセptron・ニューラルネットワーク・ディープラーニング



29

Flatten層の追加

作成したモデルに**add**で層を順番に追加していく。
ここでは最初に画像を一次元配列に変換するFlatten層を追加する。
追加したい層をインポートしておく

```
from tensorflow.keras.layers import Dense, Dropout, Flatten
model.add(Flatten(input_shape=(28, 28)))
```

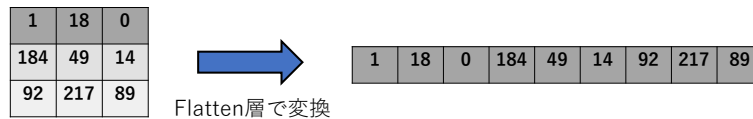
addで追加する層を指定する

一番最初の層では入力データの形状を指定する必要がある。
(ここでは28画素×28画素の2次元配列)

30

一次元配列への変換

二次元配列として保存されている画素値のデータを一次元配列に変換してからニューラルネットワークに入力する。
(二次元配列のまま入力する方法もある。)



31

中間層(隠れ層)の追加

中間層(隠れ層)として**dense**層を追加する。

Unitsで層の中のニューロンの数を指定する。
ここでは128個のニューロンを持つ層を追加する。

```
model.add(Dense(units=128, activation='relu'))
```

活性化関数としてReLU関数を使う

32

出力層の追加

出力層として、分類するクラスと同じ数のニューロンを持つdense層を追加する。

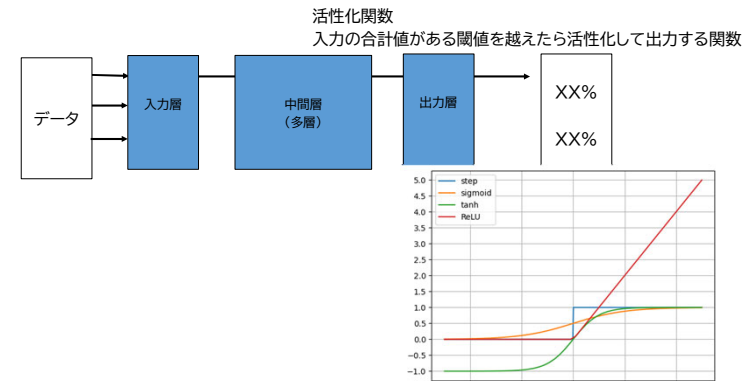
Unitsで層の中のニューロンの数を指定する。
ここでは10個のニューロンを持つ層を追加する。

```
model.add(Dense(units=10, activation='softmax'))
```

活性化関数としてsoftmax関数を使う

33

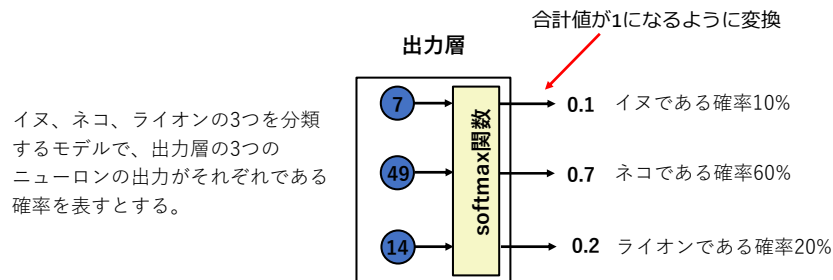
ニューラルネットワーク・ディープラーニング 活性化関数



34

Softmax関数

多クラス(2クラス以上)の分類では、活性化関数としてsoftmax関数を用いる。
softmax関数はすべての出力値の合計が1になるように、出力値を調整する。



35

シグモイド関数

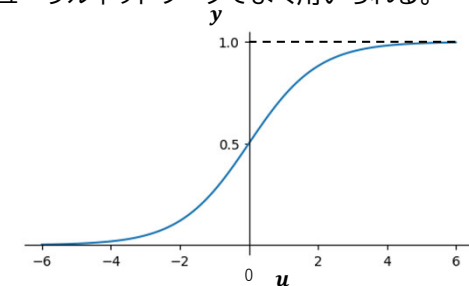
シグモイド関数は $x = 0$ で不連続に値が変化するのはなく、 $y = 0$ から $y = 1$ までの範囲を連続的に値が変化する。

微分可能であるためニューラルネットワークでよく用いられる。

シグモイド関数

$$y = f(u) = \frac{1}{1 + e^{-u}}$$

$e = 2.718 \dots$ は自然対数の底



36

ReLU関数

ReLU(Rectified Linear Unit)関数は入力値が負なら0、正なら入力値をそのまま出力する。

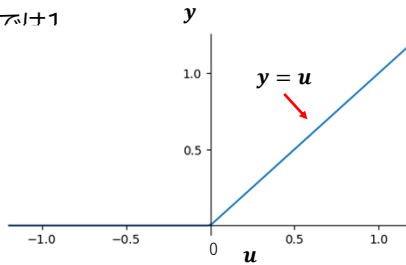
微分の値は $u < 0$ では0、 $u \geq 0$ では1

ReLU関数

$$y = f(u) = \begin{cases} u, & u \geq 0 \\ 0, & u < 0 \end{cases}$$

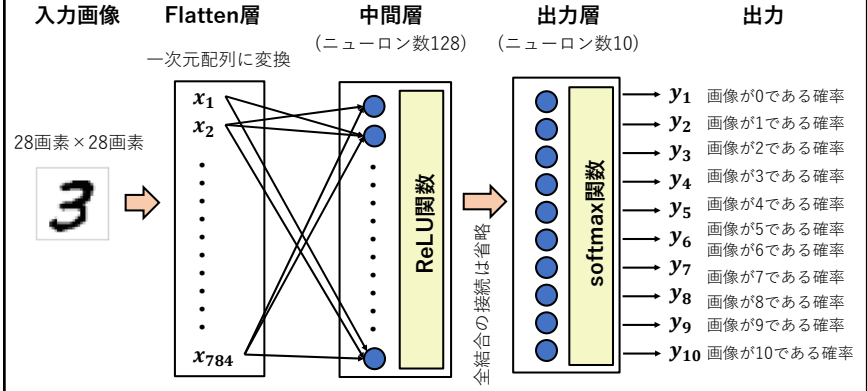
微分値の絶対値がシグモイド関数より大きい。

損失関数の勾配が小さくなりすぎず、ニューラルネットワークの層が深い場合に適している。



37

作成したニューラルネットワークの構成



38

モデルのコンパイル

層を追加したモデルで学習ができるようにするためには、モデルをコンパイルする必要がある。

損失関数を最小にするパラメータを
探すアルゴリズムの指定 (後述)

```
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
```

損失関数として交差エントロピー
誤差を用いる

学習結果の評価に使う指標を指定する。
ここでは正解率を用いる。

39

モデルの概要

`model.summary()`

summaryでモデル全体の概要を確認する。

各層の出力の数 各層のパラメータ数

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 10)	1,290

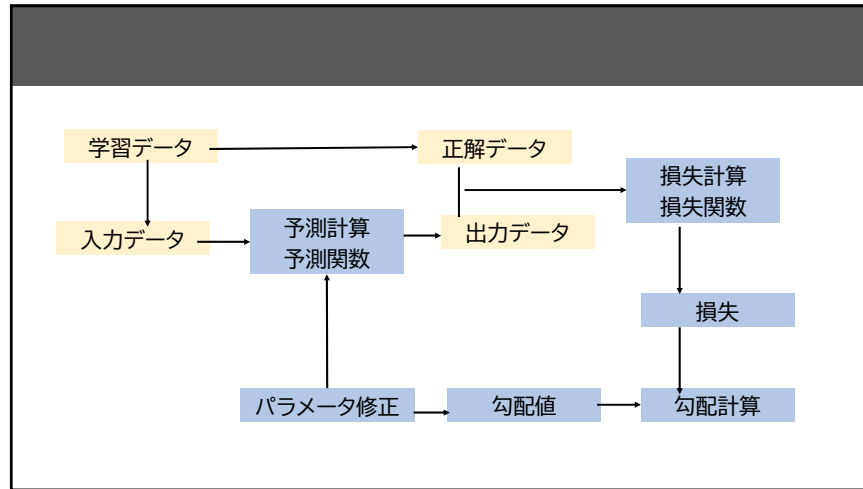
Total params: 101,770 (397.54 KB)
 Trainable params: 101,770 (397.54 KB)
 Non-trainable params: 0 (0.00 B)

784 × 128 + 128

入力値に対する重み バイアス

モデル全体で101770個ものパラメータがある

40



41

損失関数

ニューラルネットワークを使った分類モデルや回帰モデルを作るためには、正しい予測が行えるように入力値に対する重みとバイアスのパラメータを調整しなければならない。

モデルの予測とデータの実際の値とのずれを表す損失関数を導入し、損失関数の値が最小になるようにパラメータを最適化する。

42

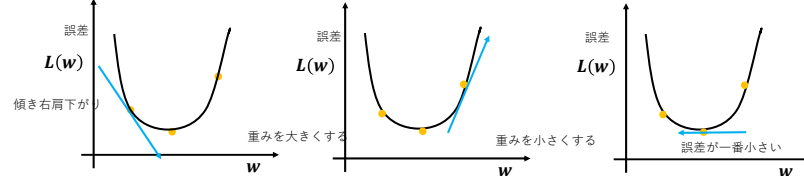
パーセプトロン・ニューラルネットワーク・ディープラーニング

誤差逆伝播法

学習するたびに正解となるものへのむずびつきを強くし、不正解とのつながりを弱める



どれだけ間違えていたか → 損失関数

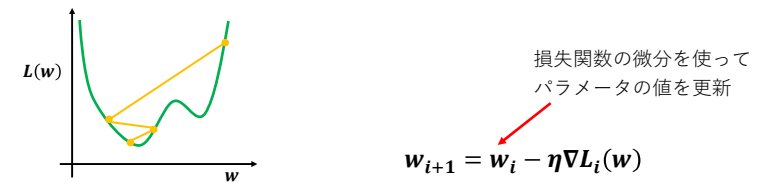


43

Adam

深層学習では扱うパラメーター数が膨大になるため、効率的に損失関数を最小にするパラメータを探す最適化アルゴリズムが必要。

最急降下法では学習率は固定されている。学習率が小さすぎると最適解に近づくのに時間がかかり、大きすぎると発散して最適解にたどり着けない。



Adamでは、以前のステップの勾配の情報も使い、学習率を(更新の幅)を動的に変えながら効率的に最適解に近づく。

44

学習の実行

訓練データを使って学習するにはfitを用いる。

訓練用のデータ **epoch数**は訓練データ全体を何回使って学習を行うかを表す。
ここでは訓練データ全体を10回使ってパラメータを調整する。

```
epochs = 10
results = model.fit(x_train, y_train, epochs=epochs, validation_data=(x_test, y_test))
```

損失関数の値の変化等、学習過程の履歴が保存される。

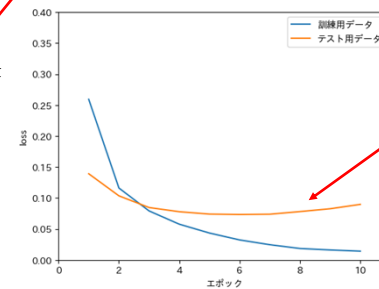
テスト用のデータを渡しておけば、テスト用のデータでも損失関数の値や成果率を計算してくれる。

45

学習履歴の表示(損失関数の値の変化)

```
ax.plot(epo, results.history["loss"], label="訓練用データ")
ax.plot(epo, results.history["val_loss"], label="テスト用データ")
```

学習過程の履歴が保存されている。
"loss"が訓練用データ、"val_loss"がテスト用データでの値。



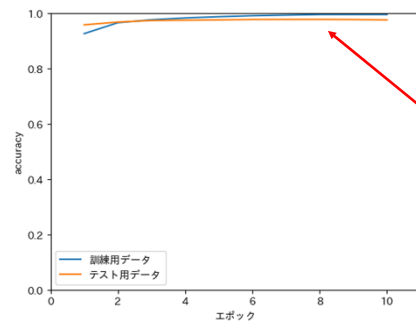
テスト用データでは損失関数の値が下げ止まっており、これ以上学習を重ねても過学習になる可能性が高い。

ネットワークの重みの初期値はランダムに設定されるため、学習のたびに結果の数値に多少の違いがでる。

46

学習履歴の表示(正解率の値の変化)

```
ax.plot(epo, results.history["accuracy"], label="訓練用データ")
ax.plot(epo, results.history["val_accuracy"], label="テスト用データ")
```



訓練用データでもテスト用データでも高い正解率を得られている。

47

学習済みモデルを用いた予測

predictに入力データを渡せば、学習済みのモデルで予測ができる。

テスト用データを使って画像の種類を表すラベルを予測させる。

```
y_pred = model.predict(x_test)
```

予測結果は、各分類ラベルが正解である確率の配列として求められる。

予測結果の例

[1.2e-12 9.5e-15 3.1e-11 1.0e-06 3.3e-18 9.8e-13 6.6e-17 9.9e-01 5.7e-13 4.1e-11]

↑
ラベルが0
である確率

↑
ラベルが1
である確率

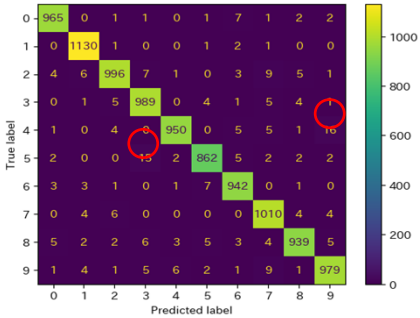
↑
ラベルが2
である確率

↑
ラベルが7である確率
(確率が最も大きい)

48

結果:混同行列の表示

混同行列を表示させれば、どの種類の画像の組み合わせが間違えやすいかなども確認できる。



9と4、3と5などの組み合わせが誤分類が多い。

49

練習問題 ファッションアイテム画像の分類

手書き数字の分類のサンプルプログラムを参考にし、ファッションアイテム(シャツ、ドレス、かばん、靴、等)の画像を分類するプログラムを実行しなさい。

ファイル『fashion_kadai.ipynb』



50

正解ラベルと画像の対応

ラベル値	画像に映っているもの
0	T-shirt/top
1	Trouser
2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot

51

52

本日の講義内容

- ニューラルネットワークによる多クラス分類
 - 損失関数、活性化関数
 - 深層学習用ライブラリTensorFlow、Keras
 - 手書き数字の画像の分類
 - 練習問題 Fashion item画像の分類
- 畳み込みニューラルネットワークによるクラス分類
 - 食事分類(CNN)
- YOLO(物体検出)
 - 食事画像から各種メニューの抽出
 - アノテーション体験

53

数字分類→食品の写真

- 出典:電気通信大学 柳井研究室 UEC FOOD 100 データセット

Yuji Matsuda and Keiji Yanai: Recognition of Multiple Food Images by Detecting Candidate Regions, Proc. of IEEE International Conference on Multimedia and Expo (ICME), (2012)

100種類の食品の写真。各食品の写真には、食品の位置を示すバウンディングボックスが付与してあるデータベース。

→演習用に、12品目、主食・主菜・副菜・汁物から選んだ

54

54

食品データ(左) 12品目(右)

	file_name	label	label_name
0	36_15803_0.jpg	1	味噌汁
1	36_15154_0.jpg	1	味噌汁
2	87_14990_0.jpg	10	グリーンサラダ
3	1_31_0.jpg	0	ごはん
4	67_14711_0.jpg	7	卵焼き
...	...	...	...
2336	36_13840_0.jpg	1	味噌汁
2337	36_14764_0.jpg	1	味噌汁
2338	1_5073_0.jpg	0	ごはん
2339	1_14834_0.jpg	0	ごはん
2340	87_13829_0.jpg	10	グリーンサラダ

2341 rows × 3 columns

	label	label_name
0	0	ごはん
1	1	味噌汁
2	2	鮭の塩焼
3	3	鶏の唐揚げ
4	4	豚カツ
5	5	ハンバーグ
6	6	豚肉の生姜焼き
7	7	卵焼き
8	8	納豆
9	9	冷奴
10	10	グリーンサラダ
11	11	豚汁

55

55

写真例



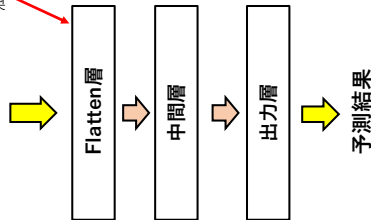
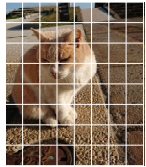
56

56

前回学んだモデルの問題点

前回の講義で学んだニューラルネットワークでは、画像をFlatten層で一次元配列に変換してから入力していた。
一次元配列でデータを入力する方法は画像以外のデータも扱えて汎用性が高いが、画素の空間的なパターンの情報が学習に反映されない。

画像を一次元
配列に変換



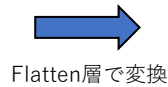
画像のフィルター処理の一種である畳み込みを利用することで、モデルを改善できる。

57

一次元配列への変換

二次元配列として保存されている画素値のデータを一次元配列に変換してからニューラルネットワークに入力する。
(二次元配列のまま入力する方法もある。)

1	18	0
184	49	14
92	217	89



1	18	0	184	49	14	92	217	89
---	----	---	-----	----	----	----	-----	----

Flatten層で変換

58

畳み込みニューラルネットワーク

畳み込みニューラルネットワーク(Convolutional Neural Network)は、**畳み込み層**と**最大値プーリング層**で画像の2次元的特徴を抽出し、それらを反映させた学習や予測を行うニューラルネットワーク。

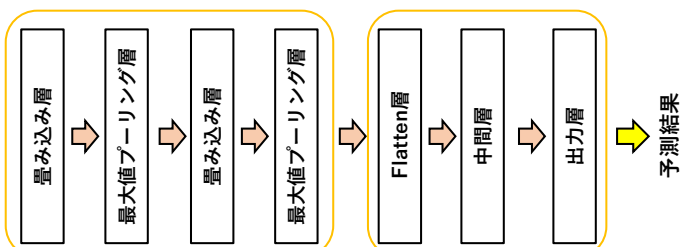
「畳み込み層 + 最大値プーリング層」
の組み合わせで特徴量マップを抽出。

特徴量マップの情報を
利用して分類・予測

手前の畳み込み
層は複数でも可



画像のまま
入力



59

畳み込み演算

カーネルの位置をずらしていきながら画像全体に対して畳み込みの演算を行うと、畳み込み後の画像(特徴量マップ)が得られる。

元画像

x_{11}	x_{12}	x_{13}	x_{14}	...	x_{1m}
x_{21}	x_{22}	x_{23}	x_{24}		
x_{31}	x_{32}	x_{33}	x_{34}		
x_{41}	x_{42}	x_{43}	x_{44}		
$\vdots$				$\ddots$	
x_{n1}		...			x_{nm}

畳み込み
畳み込み

特徴量マップ

z_{11}	z_{12}	...	$z_{1m'}$
z_{21}	z_{22}		
$\vdots$		$\ddots$	$\vdots$
$z_{n'1}$			z_{nm}

カーネルと似た画素値の
パターンを持つ領域を抽出した特徴量マップが得られる。

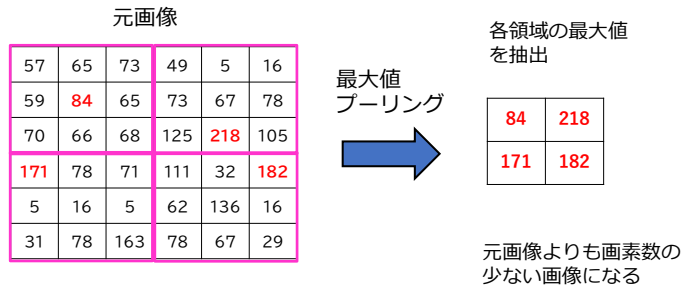
カーネルを上下左右に何画素ずつずらしていくかをストライドという

カーネルの幅の分だけ元
画像より少し小さい

60

最大値プーリング(Max Pooling)

最大値プーリングは画像を小さな領域に分割し、それぞれの領域で最も大きな値を持つ画素値を抜き出して新しい画像に変換する処理。



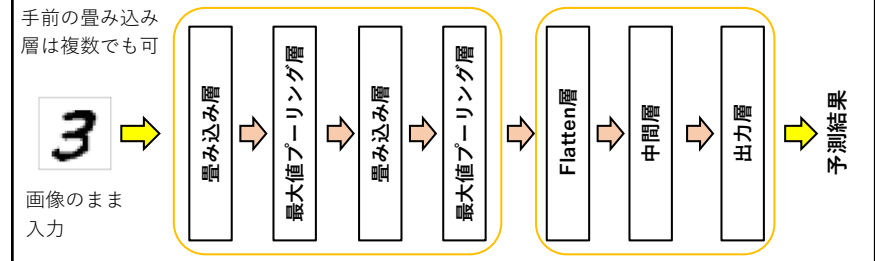
61

畳み込みニューラルネットワーク

畳み込みニューラルネットワーク(Convolutional Neural Network)は、**畳み込み層と最大値プーリング層で画像の2次元的な特徴を抽出**し、それらを反映させた学習や予測を行うニューラルネットワーク。

「畳み込み層 + 最大値プーリング層」の組み合わせで特徴量マップを抽出。

特徴量マップの情報を利用して分類・予測



62

colaboratoryでのGPUの利用

時間制限はあるが、google colaboratoryの環境ではGPUを使って計算ができる。ディープラーニングでは、GPUを使うことで大幅に実行時間を短縮できる。



63



64

CSVファイルの読み込み

pandasのデータフレームとして読み込む。

```
import pandas as pd

dir_path = 'classification_data/'

df_train = pd.read_csv(dir_path + 'train.csv')
df_test = pd.read_csv(dir_path + 'test.csv')
df_class_map = pd.read_csv(dir_path + 'class_map.csv')
```

訓練用データの
CSVファイル

テスト用データの
CSVファイル

65

CSVファイルの内容の確認

データフレームとして読み込んだ訓練用データのCSVファイルを表示する。 `df_train`
(テスト用データも同じ形式で情報が保存されている)

訓練用画像のファイル名

全部で2341 枚の訓練用画像が含まれている

	file_name	label	label_name
0	36_15803_0.jpg	1	味噌汁
1	36_15154_0.jpg	1	味噌汁
2	87_14990_0.jpg	10	グリーンサラダ
3	1_31_0.jpg	0	ごはん
4	67_14711_0.jpg	7	卵焼き
...	...	...	...
2336	36_13840_0.jpg	1	味噌汁
2337	36_14764_0.jpg	1	味噌汁
2338	1_5073_0.jpg	0	ごはん
2339	1_14834_0.jpg	0	ごはん
2340	87_13829_0.jpg	10	グリーンサラダ
2341 rows x 3 columns			

各画像の正解ラベル

66

画像データの読み込み

```
from keras.preprocessing.image import load_img, img_to_array
import numpy as np

def load_images(df):
    images = []
    labels = []
    for _, row in df.iterrows():
        fname = dir_path + 'images/' + row['file_name']
        img = load_img(fname)
        img = img_to_array(img)
        images.append(img)
        labels.append(row['label'])
    return np.array(images), np.array(labels)

x_train, y_train = load_images(df_train)
x_test, y_test = load_images(df_test)
```

Kerasにはファイルから画像を読み込むための命令として **load_img**、画像を配列に変換する命令として **img_to_array** が用意されている。

リストされている画像ファイルを、for文で順番に読み込んでいく。

深層学習の入力データとして使えるように、画像データを多次元配列に変換する。

67

データのサイズと次元の確認

numpyの多次元配列はshapeで各次元の要素数を確認できる。

```
print("訓練用画像の枚数、画素数、チャンネル数")
print(x_train.shape)
print("テスト用画像の枚数、画素数、チャンネル数")
print(x_test.shape)
```

訓練用画像は
2341枚

テスト用画像は
585枚

訓練用画像の枚数、画素数、チャンネル数

(2341, 128, 128, 3)

テスト用画像の枚数、画素数、チャンネル数

(585, 128, 128, 3)

画像は画素数128×128、
チャンネル数3(R,G,B)の
カラー画像

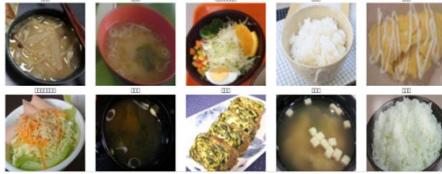
68

画像を表示

```
import matplotlib.pyplot as plt

class_names = dict(zip(df_class_map['label'], df_class_map['label_name']))

fig, axes = plt.subplots(2, 5, figsize=(15, 6))
for i, ax in enumerate(axes.flat):
    ax.imshow(x_train[i].astype('uint8'))
    ax.set_title(class_names[y_train[i]])
    ax.axis('off')
plt.tight_layout()
plt.show()
```



matplotlibの`imshow`を使うと
配列を画像として表示できる

69

ラベルのone-hotエンコーディング

```
from tensorflow.keras.utils import to_categorical
```

```
num_classes = len(df_class_map)
```

```
y_train_cat = to_categorical(y_train, num_classes)
```

```
y_test_cat = to_categorical(y_test, num_classes)
```

70

70

モデルの構築(CNN)

Kerasのsequentialモデルを作成する。
Sequentialをインポートしておく

```
from tensorflow.keras.models import Sequential

model = Sequential()
```

Sequentialでモデルを作成する。
この時点ではまだニューロンの層がない空っぽの状態。

71

畳み込み層の追加 (1)

作成したモデルに**add**で層を順番に追加していく。

最初に画像の畳み込み(コンボリューション)を行うConv2D層を追加する。

畳み込みに用いる**フィルター**

(カーネル)の数を指定。

ここでは32個とした

カーネルの大きさの指定。

ここでは3画素×3画素とした。

```
from tensorflow.keras.layers import Conv2D

# 畳み込み層(1)
model.add(Conv2D(filters=32, kernel_size=(3, 3),
                  activation='relu', input_shape=(128, 128, 3)))
```

活性化関数としてReLU関数を使う

一番最初の層では入力データの形状を指定する。

(128画素×128画素、3チャンネル(R,G,B))

一次元配列に変換せずに入力している

72

畳み込み層の追加 (2)

今回作成するモデルでは、特徴量マップに対してさらに畳み込みを行う。

畳み込みに用いるフィルター(カーネル)の数を指定。
この層では64個とした

```
# 畳み込み層(2)
model.add(Conv2D(filters=64, kernel_size=(3, 3),
                  activation='relu'))
```

他のパラメータは1層目の畳み込み層と同じにしている。

73

最大値プーリング層の追加

畳み込み層の後ろに最大値プーリングを行うMaxPooling2D層を追加する。

事前にインポートしておく

```
from tensorflow.keras.layers import MaxPooling2D

model.add(MaxPooling2D(pool_size=(2, 2)))
```

プーリングを行う小領域の大きさの指定。
ここでは2画素×2画素の幅で最大値プーリングを行う。

74

二組目の畳み込み層とプーリング層の追加

今回作成するモデルでは、ここまで追加した畳み込み層と最大値プーリング層と同じ構成の層をもう一組追加する。

```
model.add(Conv2D(filters=32, kernel_size=(3, 3),
                  activation='relu'))

model.add(Conv2D(filters=64, kernel_size=(3, 3),
                  activation='relu'))

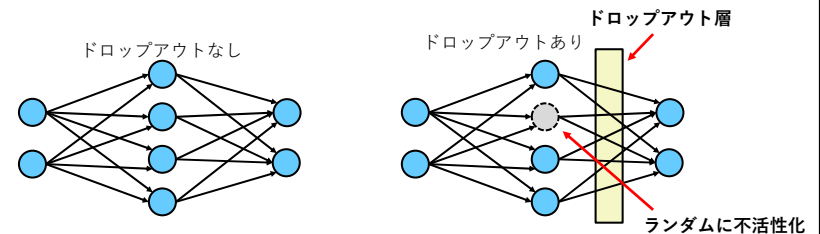
model.add(MaxPooling2D(pool_size=(2, 2)))
```

75

ドロップアウト

ニューラルネットワークが訓練用データに対して最適化されすぎると過学習の状態になり、訓練用データ以外に対する予測精度が低下してしまう。

ドロップアウトは深層学習における過学習を抑制する手法のひとつで、ニューロンからの出力を一定の割合でランダムに不活性化(出力を無視)する。



76

ドロップアウト層の追加

ドロップアウトを行う**Dropout**層を追加する

事前にインポートしておく

```
from tensorflow.keras.layers import Dropout
model.add(Dropout(0.25))
```

不活性化するニューロンの割合を指定する。
ここでは25%とした。
0.2~0.5程度の範囲で設定

77

後半部分の層の追加

畳み込み層と最大値プーリング層から得られた**特徴量マップ**から
分類予測を行うための層を追加していく。

特徴量マップを一次元配列
に変換する

```
model.add(Flatten())
model.add(Dense(units=128, activation='relu'))
model.add(Dropout(0.25))
```

出力層の前にもドロップ
アウト層を追加

特徴量マップの情報で分類
するための中間層

78

出力層の追加

二つのクラスを分類するための出力層として、ニューロンを一つだけの**dense**層を追加する。

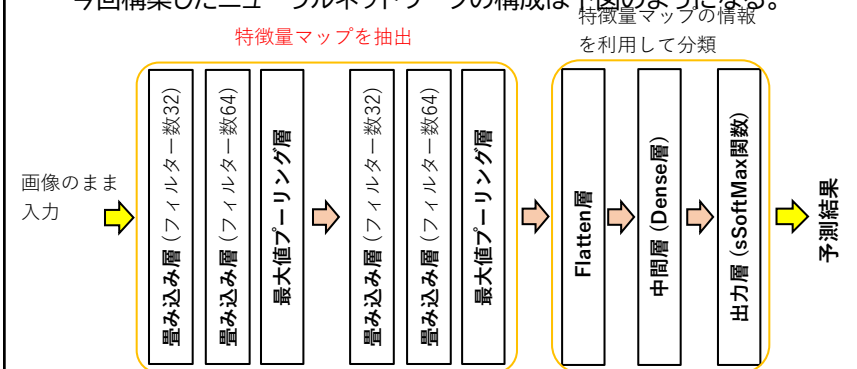
```
# 出力層: 12クラス分類なのでsoftmax
model.add(Dense(units=num_classes, activation='softmax'))
```

12クラス分類なので、**softmax関数**を活性化関数として使う

79

構築したニューラルネットワークの構成

今回構築したニューラルネットワークの構成は下図のようになる。



80

モデルのコンパイル

層を追加したモデルで学習ができるようにするためには、モデルをコンパイルする必要がある。

損失関数を最小にするパラメータを
探すアルゴリズムの指定

```
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
```

損失関数として交差エントロピー誤差を用いる。

多クラス(3クラス以上)の場合は
categorical_crossentropyを指定する。

学習結果の評価に使う指標を
指定する。
ここでは正解率を用いる。

81

学習の実行

訓練データを使って学習するにはfitを用いる。

epoch数は訓練データ全体を何回使って学習を行うかを表す。
ここでは訓練データ全体を10回使ってパラメータを調整する。

訓練用のデータ

```
epochs = 10
results = model.fit(x_train, y_train, epochs=epochs,
                    validation_data=(x_test, y_test))
```

損失関数の値の変化等、学習
過程の履歴が保存される。

テスト用のデータを渡しておけば、
テスト用のデータでも損失関数の
値や成果率を計算してくれる。

82

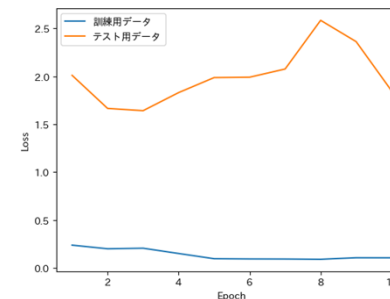
```
epo = range(1, epochs + 1)
```

```
fig, ax = plt.subplots()
ax.plot(epo, results.history["loss"], label="訓練用データ")
ax.plot(epo, results.history["val_loss"], label="テスト用データ")
ax.set_xlabel("Epoch")
ax.set_ylabel("Loss")
ax.legend()
plt.show()
```

83

83

学習履歴の表示(損失関数の値の変化)



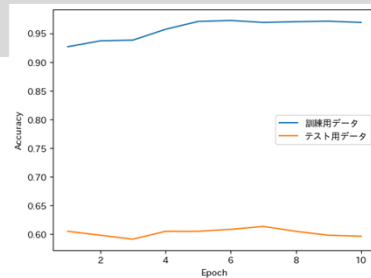
このモデルは典型的な過学習を示します(訓練Accuracy~97%、検証Accuracy~60%で頭打ち、
訓練Lossはほぼ0まで下がるのに検証Lossはepoch3以降増加)。

ネットワークの重みの初期値はランダムに設定されるため、学習のたびに
結果の数値に多少の違いがある。(講義のスライドと全く同じにはならない)

84

学習履歴の表示(正解率の値の変化)

```
fig, ax = plt.subplots()
ax.plot(epo, results.history["accuracy"], label="訓練用データ")
ax.plot(epo, results.history["val_accuracy"], label="テスト用データ")
ax.set_xlabel("Epoch")
ax.set_ylabel("Accuracy")
ax.legend()
plt.show()
```



このモデルは典型的な過学習を示します(訓練Accuracy~97%、検証Accuracy~60%で頭打ち、

訓練Lossはほぼ0まで下がるのに検証Lossはepoch3以降増加)。

85

学習済みモデルを用いた予測

predictに入力データを渡せば、学習済みのモデルで予測ができる。

テスト用データを使って画像の種類を表すラベルを予測させる。

```
y_pred = model.predict(x_test)
y_pred_label = np.argmax(y_pred, axis=1)
```

86

```
# テスト画像といくつかの予測結果を表示
fig, axes = plt.subplots(2, 5, figsize=(15, 6))
for i, ax in enumerate(axes.flat):
    ax.imshow(x_test[i].astype('uint8'))
    correct = "○" if y_pred_label[i] == y_test[i] else "×"
    ax.set_title(f"{correct} 予測: {class_names[y_pred_label[i]]} \n 正解: {class_names[y_test[i]]}", fontsize=9)
    ax.axis('off')
plt.tight_layout()
plt.show()
```

87

87

テスト用画像と予測結果の一部を表示

テスト用画像の一部とモデルの予測結果を表示してみる。
ここに示したものはほとんど正しく予測できている。



88

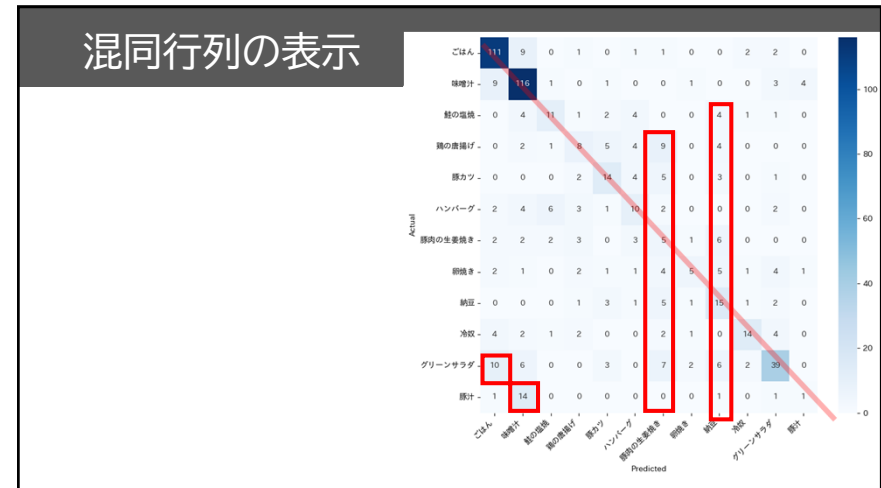
Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 126, 126, 32)	896
conv2d_1 (Conv2D)	(None, 124, 124, 64)	18,496
max_pooling2d (MaxPooling2D)	(None, 62, 62, 64)	0
conv2d_2 (Conv2D)	(None, 60, 60, 32)	18,464
conv2d_3 (Conv2D)	(None, 58, 58, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 29, 29, 64)	0
flatten (Flatten)	(None, 53824)	0
dense (Dense)	(None, 128)	6,889,600
dropout (Dropout)	(None, 128)	0
flatten_1 (Flatten)	(None, 128)	0
dense_1 (Dense)	(None, 128)	16,512
dropout_1 (Dropout)	(None, 128)	0
dense_2 (Dense)	(None, 12)	1,548

Total params: 6,964,012 (26.57 MB)
 Trainable params: 6,964,012 (26.57 MB)
 Non-trainable params: 0 (0.00 B)

89

89



90

モデルの修正 food.CNN_classification_自己学習版.ipynb

- 今のCNNのまま、学習のやり方を改善する(データ拡張・EarlyStopping・クラス不均衡対策)
- 事前学習済みモデル(転移学習)に土台を差し替える(一番効果が大きいはず)

実行前に予想してみましょう:

さきほどの混同行列では、味噌汁と豚汁が頻繁に混同されていました。データ拡張・class_weight・転移学習を加えると、この混同は直ると思いますか？それとも、データ量を増やしても解決しない種類の混同だと思いますか？実行後、自分の予想と結果を比べてみてください。

91

91

12. 精度改善を試してみよう

- ここまでで「実行→解釈」は一通り終わりました。ここからは、実際に精度を上げられるか試す時間です。効果が大きい順に、2つのやり方を用意しています。両方試して、混同行列がどう変わるか比べてみましょう。
- **12-1:** 今のCNNのまま、学習のやり方を改善する(データ拡張・EarlyStopping・クラス不均衡対策)
- **12-2:** 事前学習済みモデル(転移学習)に土台を差し替える(一番効果が大きいはず)

92

92

12-1: 今のCNNのまま、学習のやり方を改善

- データ拡張: 回転・反転・拡大でデータのバリエーションを増やす
- EarlyStopping: val_lossの改善が止まったら自動で学習を打ち切る
- class_weight: 件数の少ないクラス(生姜焼きなど)を軽視しないようにする

93

93

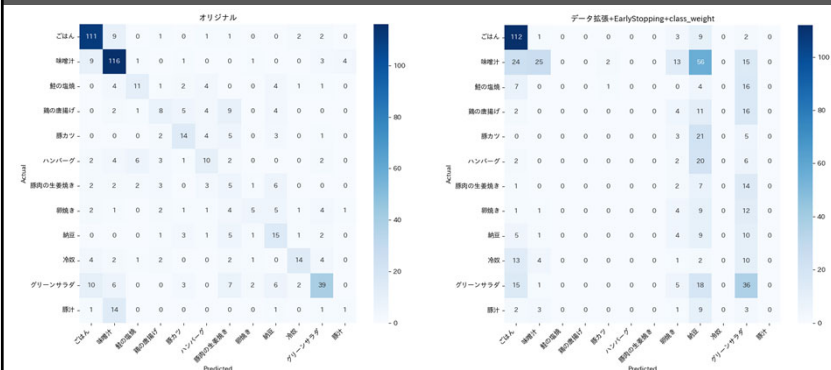
12-2. 転移学習(事前学習済みモデル)に差し替える

- ImageNetで事前学習済みのMobileNetV2を土台にする

94

94

前回 → 12-1



95

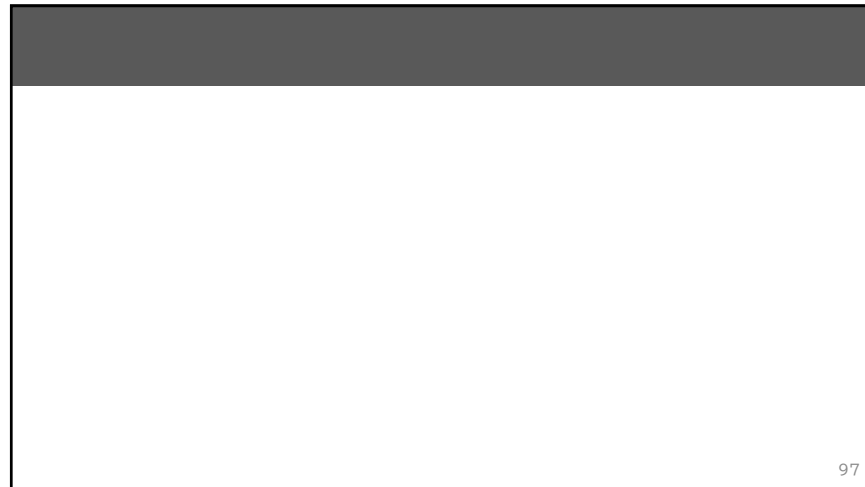
95

前回 → 12-2



96

96



97

97

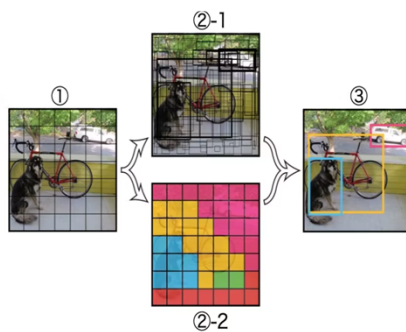
1枚の写真に複数の品目が写っている場合の判別



98

98

物体検出 YOLO



①マス目に分解

②-1 バウンディングボックスを配置

信頼度が0に近いほど、バウンディングボックスの中身は「背景」を表しています。
 ・信頼度が1に近いほど、バウンディングボックスの中身は「物体」を表しています。

②-2 何を表しているか区分

③ ②より判断

99

以下のファイルを使用する

- 先ほどの100種類のデータを使用して、あらかじめ学習してアルモデルを用いる

- 学習済みモデルを使用した演習を行う

<使用データ>

- 「food_YOLO_exercise.ipynb」
- 「best.pt」
- 検証データファイル

100

100

food_YOLO_exercise.ipynb

food_YOLO_exercise.ipynb

ファイル 編集 表示 挿入 ランタイム ツール ヘルプ

Q コマンド + コード + テキスト | すべてのセルを実行 | ドライブにコピー

Food Detection Exercise (学生用)

学習済みのYOLOモデルを使って、配布された写真から複数の食品を検出してみましょう。学習は済んでいるので、このノートブックではモデルを読み込んで、推論だけです。

+ コード + テキスト

1. モデルと写真の準備

```
!pip install gdown -q
# ↓ 請願用Google Driveに置いた food_yolo_exercise.zip のファイルIDに書き換えてください
FILE_ID = "15190FPBrsuXGincRBJ11Y0NEUL-0K5"
!gdown --id (FILE_ID) -O food_yolo_exercise.zip
!unzip -q -o food_yolo_exercise.zip -d .
```

2. 学習済みモデルの読み込み

best.pt (先生が事前に用意した、12品目・学習済みの重み)を読み込みます。

101

```
# 配布された写真の1枚を試してみる(ファイル名は実際のものに書き換えてください)
#result = model.predict(photo_dir + "karaage2.png", conf=0.01, verbose=False)[0]
result = detect_and_show_yolo(photo_dir + "karaage2.png", confidence=0.02)
```

result = detect_and_show_yolo(photo_dir + "karaage2.png", confidence=0.02)

ごはん: confidence = 0.92 味噌汁: confidence = 0.83 グリーンサラダ: confidence = 0.68 鶏の唐揚げ: confidence = 0.23 グリーンサラダ: confidence = 0.05 グリーンサラダ: confidence = 0.05 ごはん: confidence = 0.02 鶏の唐揚げ: confidence = 0.02 グリーンサラダ: confidence = 0.02 グリーンサラダ: confidence = 0.01

赤字部分を判定したいファイルに変更して実行する

102

ごはん: confidence = 0.87
豚汁: confidence = 0.83
納豆: confidence = 0.53
グリーンサラダ: confidence = 0.06

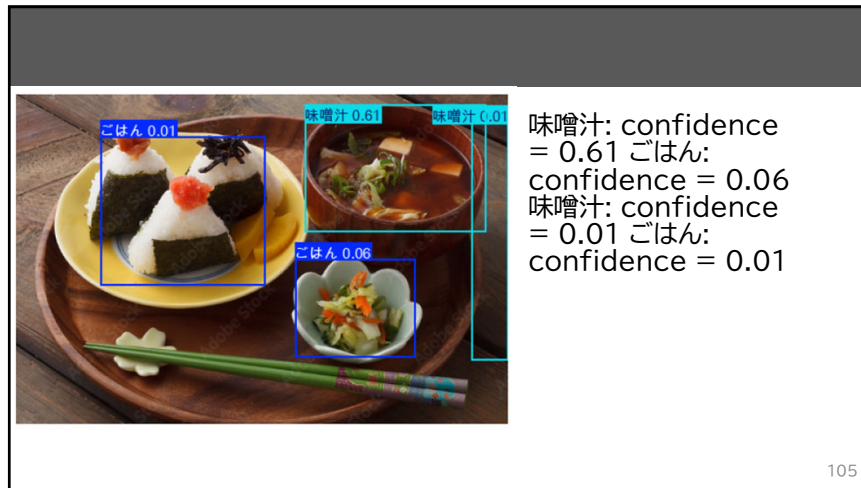
103

103

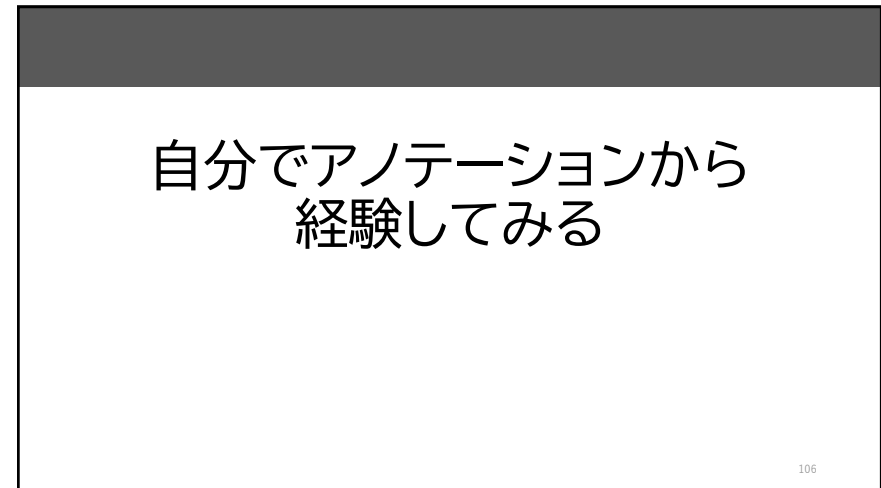
ごはん: confidence = 0.92
味噌汁: confidence = 0.82
鮭の塩焼: confidence = 0.62
グリーンサラダ: confidence = 0.54

104

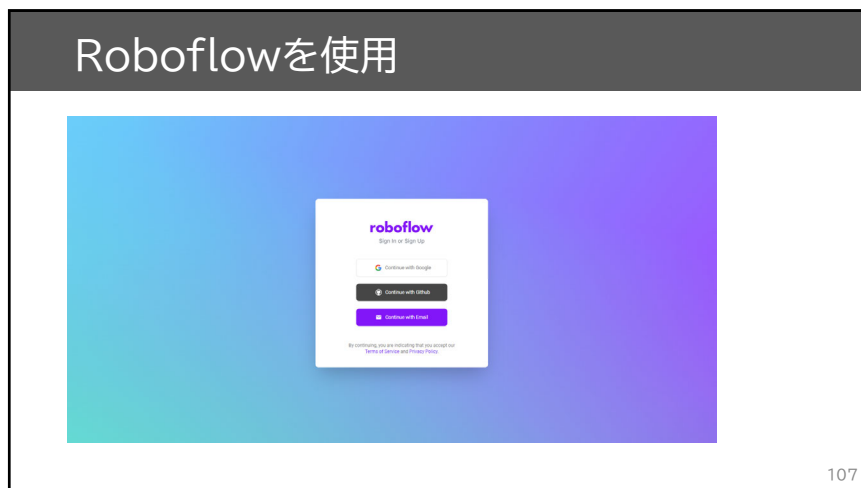
104



105



106

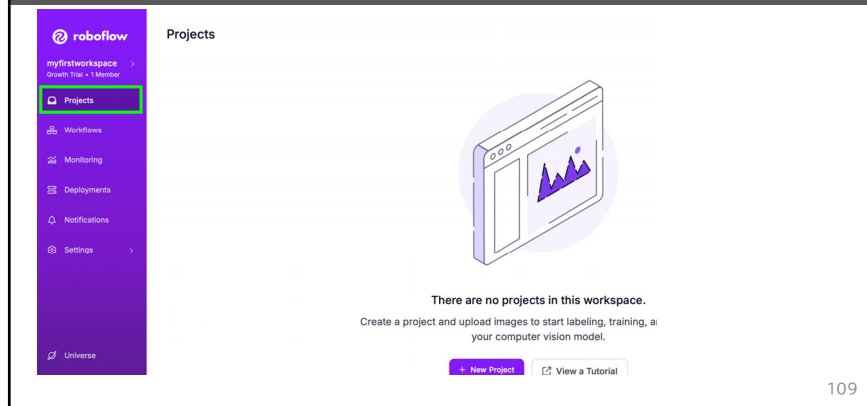


107



108

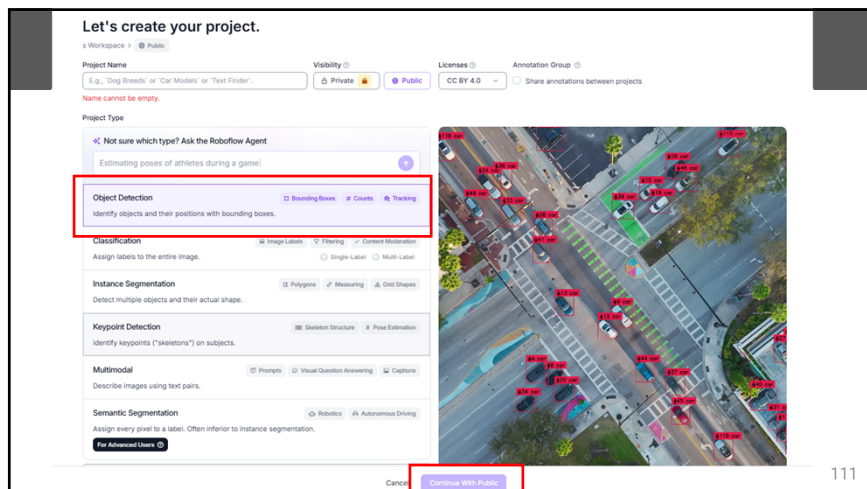
プロジェクト作成



109

モデル	区分	説明	例
Object Detection	物体検出	画像内の対象物の種類と位置を認識する	画像内から犬を検出
Single-Label Classification	シングルラベル画像分類	対象の画像に一つのラベルを付け識別する	良品写真 or 不良品写真
Multi-Label Classification	マルチラベル画像分類	対象の画像に複数のラベルを付け識別する	画像に犬と猫が写っていることを識別
Instance Segmentation	インスタンスセグメンテーション	画像内に映っている対象物を種類毎に個別に認識し領域を抽出する	自動運転などで車・人を個体毎に識別
Keypoint Detection	キーポイント検出	画像内に映っている対象物の向きを識別する	骨格検知、顔の向きの検知
Multimodal	マルチモーダル	画像を表現する区分・内容を定義し画像からテキストを生成する	画像からテキストを生成し、画像の説明等を行う

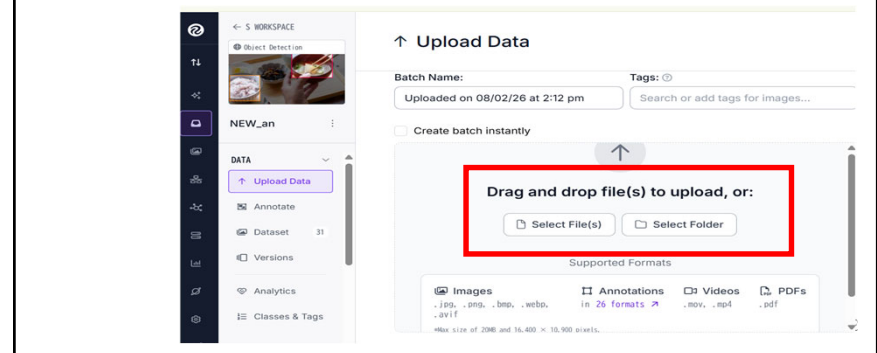
110



111

画像ファイルをアップ

- Ann_1.zip (31枚)



112

全てファイルをアップ→保存する

↑ Upload Data

Batch Name: Tags:

☐ Create batch instantly

All Images 8 Annotated 0 Not Annotated 8

Drag and drop images, annotations, and videos.
 .jpg, .png, .bmp, .webp, .avif in 26 formats [View formats](#) ☐ .mov, .mp4
 Max size of 20MB and 16,400 × 10,900 pixels.

保存する

113

アノテーション作業

Unannotated 8 Annotated 0 Sort By Newest

asa2.png gyozuza.png gyozuza2.png harumaki.png Deli.png iroiro.png

asa.png himawari.png

114

①□のカーソルを選択し、②食品を囲み、③食品名を選択

Annotation Editor

③ 餃子

1 Egg 2 Gyoza 3 Gyoza 4 Men 5 Miso 6 Natto 7 NEW-an 8 Niku 9 Rice

Options ▾

②

115

アノテーション作業が終わったら→学習

Train a Model

Select Engine

Select how you want to train your model. You can either configure the training yourself or let Roboflow find the optimal model automatically.

☒ Custom Training ①

Choose your model architecture and configure training parameters.

- Select from existing model architectures
- Configure model size and checkpoints

Select Architecture

Select the model architecture and size that best fit your use-case. Smaller sizes run faster, but may have lower accuracy.

☒ Roboflow RF-DETR ②

- Fast, real-time model. Highest accuracy on COCO
- Needs less data and converges earlier. Slower training

Model Size:

YOLO26

- Latest from Ultralytics
- End-to-end MMS-free inference
- Faster CPU inference

Model Size:

Roboflow 3.0 YOLOv11

116

学習開始

Search versions... ☐ Hide Unavailable

v1 2026-08-02 2:06pm 31 Roboflow Instant 原田重紀子

24 Train 4 Valid 3 Test

Auto-Orient Reuse Settings in a New Version

ready to train
Train Roboflow RF-DETR on v1 with 31 images

Roboflow RF-DETR (Small) Version 1 - 31 images 24 train / 4 valid / 3 test 512x512 (Stretch to)

Estimated Roboflow Credits: 1.03 [View Usage](#)

Estimated Training Time: 23 minutes - 31 minutes
Lower bound assumes typical early stopping; upper bound runs all 100 epochs.

Select Engine — Select Architecture — Select Version **Start Training**

117

NAME	UPDATED	METRICS	TYPE	DATASET VI
-9osjl-3-rfdetr-small-t1	July 27, 2026 9:47 PM	Valid Set: 61.0% Precision: 64.4% Recall: 47.6% F1: 54.7%	Roboflow RF-DETR Object Detection (Small)	menu-det

Test Set  [View Test Set ->](#)

Drop Files or [+ Select Files](#)

[Image or Youtube URL...](#)

Try with:
[My Webcam](#)
[My Machine](#)

118

118

検証データ(Ann2-4.zip)で判定してみる

Test Set  [View Test Set ->](#)

ここからファイルアップ **Drop Files or + Select Files**

[Image or Youtube URL...](#)

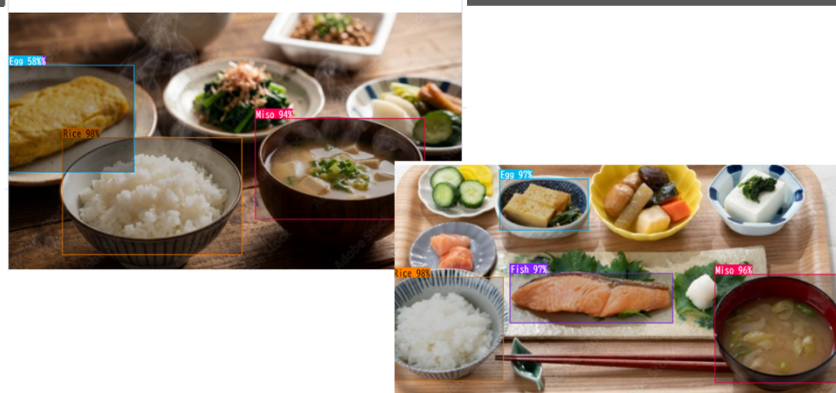
Try with:
[My Webcam](#)
[My Machine](#)



19

119

検証



120

120

アノテーション→学習が間に合わない場合

- 既存の学習モデルで試してみる（学習済みモデル.txt）

「new_an-ch7ze-1-rfdetr-small-t1」モデル

- <https://app.roboflow.com/s-workspace-2o1en/new-an-ch7ze/models/s-workspace-2o1en/new-an-ch7ze-1-rfdetr-small-t1>

「-9osjl-3-rfdetr-small-t1」モデル

- <https://app.roboflow.com/s-workspace-2o1en/-9osjl/models/s-workspace-2o1en/-9osjl-3-rfdetr-small-t1>

121

121

畳み込みニューラルネットワークの応用

今回の講義ではそれぞれの画像に何が写っているかだけで分類したが、1枚の画像（動画も含む）の中のどこに何が写っているかを検出する物体検知、特定のものが写っている領域を画素単位で分割する（塗分ける）セマンティックセグメンテーション等、応用範囲は広い。

物体検知

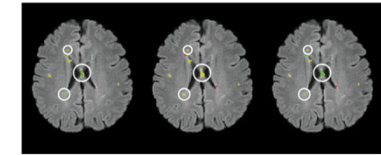
車の自動運転での利用



Nvidia社ブログ <https://blogs.nvidia.co.jp/blog/> より

セマンティックセグメンテーション

脳のMRI画像からの病変の検出



T. Brosch et al. 2016より

122